

Taming State-Space Explosion in Cloud Accounts

Finding Privilege Escalation at Scale

Claudia Cauli

Principal Research Engineer & Formal Methods Team Lead
Cloud Reliability Lab – Huawei Ireland Research Centre

FLoC 2026 Mentoring Workshop
25 July 2026

Not a lecture on Cloud security

A real security problem solved with SMT, abstraction, and simulation.

PART I: What is this about?

- What is a privilege escalation?
- Why is it hard?
- What's the naive solution?

PART II: How did FM help?

- Let the problem drive the solution
- Textbook theory pays off
- Proofs for insight, beyond assurance

Did it work?

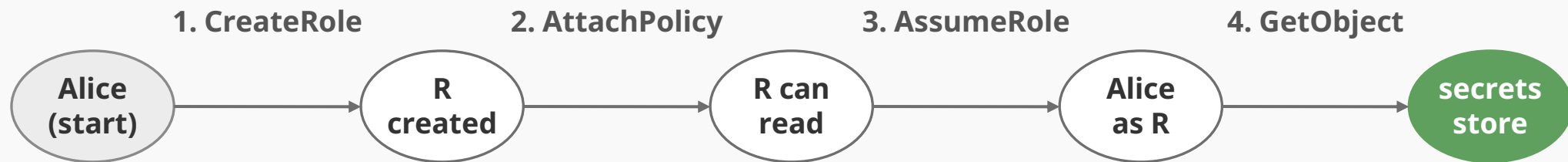
PART I

What is this about?

TL;DR: *a big security problem, and a very practical solution.*

What is a privilege escalation? aka PE

- *A path through which an identity accesses a resource that they were not meant to access.*
- Identity & Access Management (IAM): Policies, Users, Roles, Groups (Entities).
- We focus on bounded privilege escalations (BPE).

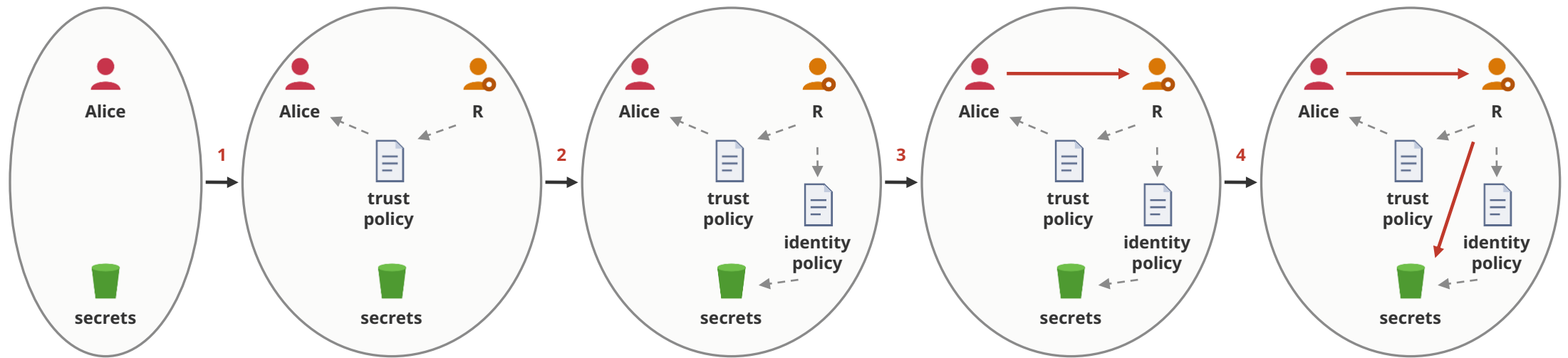


Steps 1–2 mutate the account's configuration, preparing for the escalation. Step 3 escalates Alice's permissions. Step 4 traverses the escalated state.

$$\neg \text{access}(id, res) \wedge \text{EF}_{\leq n} \text{access}(id, res)$$

Why is it hard? The attacker rewrites the system

- Dynamically changing IAM transition system.
- Mutating actions, by definition, mutate the IAM configuration.
- Bounded reachability on a self-modifying transition relation.
- Planning problem whose solution is a witness escalation sequence.



The IAM Transition System

- **S** IAM configurations (entities, policies, attachments) paired with a set of held credentials.
- $s_0 \in S$ the customer's account IAM configuration, plus the credentials of the initial principal.
- **Act** 19 mutating IAM actions.
- **Req** parameterised action calls: an action from Act together with its arguments.
- $\rightarrow \subseteq S \times \text{Req} \times S$ a step exists iff the request is authorised, and its effect on the configuration is prescribed per action.

```

4
5  /-- The 19 mutating actions. -/
6  ∨ inductive IAMAction where
7    | addUserToGroup | removeUserFromGroup
8    | attachPolicyToUser | attachPolicyToGroup
9    | detachPolicyFromUser | detachPolicyFromGroup
10   | updateLoginProfile
11   | createUser | createGroup | createPolicy
12   | updateUserInfo | updateGroupInfo | updatePolicyContent
13   | assumeRole | attachPolicyToRole | detachPolicyFromRole
14   | createRole | updateRoleInfo | updateTrustPolicy
15   deriving DecidableEq
16
17  ∨ /-- A state is an IAM configuration with users, groups, roles, policies
18    and their attachments; together with the credentials currently known. -/
19  ∨ structure State where
20    config : Configuration
21    credentials : Set ID
22    credentials_subset : credentials ⊆ IDs config.users ∪ IDs config.roles
23
24  ∨ /-- A request is authorized in `s` when its principal is credentialed and its
25    attached policies grant it (full definition elsewhere). -/
26    opaque AuthZ : State → Set Request
27
28  ∨ /-- An IAM transition system. A step is one authorized request,
29    `step : State → Request → State` that exists exactly when the request is
30    authorized (authorization and the transition relation are the same thing)
31    and each of the 19 actions carries a spec prescribing how it mutates the
32    configuration (one per action, not here). -/
33  ∨ structure IAMTransitionSystem where
34    step : State → Request → State → Prop
35    auth_spec : ∀ (σ : State) (r : Request),
36      (∃ τ : State, step σ r τ) ↔ r ∈ AuthZ σ

```

What's the naive solution? SMT-based BMC

- Shevrin & Margalit: *“Detecting Multi-Step IAM Attacks in AWS Environments via Model Checking”*.
- SMT Bounded Model Checking (BMC) with a bound of $n = 5$. Hit a scalability wall.
- 10 s timeout: found 1-step BPEs in accounts with ~100 entities, 3-step BPEs in accounts with ~30 entities.
- Huawei Cloud VIP-5 customers' accounts are far larger.

Dataset	# Entities	# Policies	# Policy Attachments	# Policy Statements*	Baseline (s)
Test Account 1	25	655	6	2978	144
Test Account 2	155	174	155	311	203
Test Account 3	192	42	38	216	131
Test Account 4	210	150	247	374	332
Test Account 5	746	922	2351	3425	3160
Test Account 6	1263	87	76	132	3000
Test Account 7	1612	546	3015	762	OOM

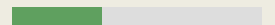
* IAM Policy statements carry logical conditions and string regexes.
SMT is the right tool for the job (a story for another time...).



PART II

How did FM help?

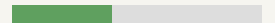
TL;DR: *immensely; made the difference between launching and not launching the feature.*



LESSON 1 OF 3

Let the problem drive the solution

Don't always shoot for the strongest results. Bounded, existential, soundness-first work.



Three decisions

- **Bounded** (≤ 5 steps) Security engineers' preference.
- **Existential** Security engineers need to find attacks.
- **Soundness-first** Never missing a real escalation is key.

De-prioritise proving the strongest result:
absence of escalation at any number of steps.

```

5  ∨ def AuthZ (s : State) : Set Request :=
6    {r : Request |
7      ∃ p : Principal,
8        principalMem p s.config ∧
9        Principal.id p ∈ s.credentials ∧
10       r var_principal = p ∧
11       r ∈ Granted (policiesAttachedToPrincipal p s.config) ∧
12       (r var_action = IAMAction.assumeRole →
13         ∃ a' ∈ s.config.roles, a'.id = (r var_role).id ∧
14         r ∈ ((eval a'.trust : Set Request)))}
15
16  ∨ inductive reachable_n_in (TS : IAMTransitionSystem) (S : Set Request) :
17    Nat → State → State → Prop
18    | refl (s) : reachable_n_in TS S 0 s s
19    | step {n s u t r} :
20      r ∈ S →
21      TS.step s r u →
22      reachable_n_in TS S n u t →
23      reachable_n_in TS S (n+1) s t
24
25  ∨ def BPE (TS : IAMTransitionSystem) (n : Nat) (S : Set Request) :
26    Set (State × Set Request) :=
27    { (s, R) | centeredOn R (allIDs s.config) ∧
28      ∃ t m, m ≤ n ∧ reachable_n_in TS S m s t ∧ ∃ r ∈ R, r ∈ AuthZ t }
29
30  ∨ def PE (TS : IAMTransitionSystem) (S : Set Request) :
31    Set (State × Set Request) :=
32    { (s, R) | ∃ m, (s, R) ∈ BPE TS m S }
33

```

Set a goal. Early

- We needed to make it faster. How fast?
- We needed to make it scale. How much?
- *Can't decide on a solution without considering the problem's constraints.*
- Our partner IAM team pulled aggregated anonymised customer stats.
- Customer tiers from VIP-0 to VIP-5: largest customers who are most in need of this feature.

<15 s for p99.9 of VIP-5 accounts

LESSON 2 OF 3

Textbook theory pays off

Build solid foundations. Everything else can be done by engineers (and LLMs).



The reduction, intuitively

- Cloud accounts are huge but massively repetitive: 1000 employees? 1000 (nearly) identical IAM users.
- Don't verify the full account. Verify an abstraction of it.
- Partition entities into equivalence classes: then either:
 - keep a few representatives, or
 - merge all members into one.
- Equivalence keys need to be carefully chosen to guarantee BPE-preservation.

```

7
8 ∨ def UserEquiv (σ : State) (u v : User) : Prop :=
9   u ∈ usersOf σ ∧ v ∈ usersOf σ ∧
10  (∀ p ∈ policiesOf σ, PolicyStableUnderNameRenaming p (nameSwap u.info v.info)) ∧
11  (∀ p ∈ policiesOf σ, PolicyStableUnderRenaming p (swapIDs u.id v.id)) ∧
12  policiesDirectlyAttachedToUser u σ.config = policiesDirectlyAttachedToUser v σ.config ∧
13  groupsUserIsLinkedTo u σ.config = groupsUserIsLinkedTo v σ.config ∧
14  (u.id ∈ credsOf σ ↔ v.id ∈ credsOf σ)
15
16 ∨ def GroupEquiv (σ : State) (g h : Group) : Prop :=
17   g ∈ groupsOf σ ∧ h ∈ groupsOf σ ∧
18   (∀ p ∈ policiesOf σ, PolicyStableUnderGroupNameRenaming p (groupNameSwap g.info h.info)) ∧
19   (∀ p ∈ policiesOf σ, PolicyStableUnderRenaming p (swapIDs g.id h.id)) ∧
20   policiesAttachedToGroup g σ.config = policiesAttachedToGroup h σ.config ∧
21   usersInGroup g σ.config = usersInGroup h σ.config
22
23 ∨ def RoleEquiv (σ : State) (a a' : Role) : Prop :=
24   a ∈ rolesOf σ ∧ a' ∈ rolesOf σ ∧
25   (∀ p ∈ policiesOf σ, PolicyStableUnderRoleNameRenaming p (roleNameSwap a.info a'.info)) ∧
26   (∀ p ∈ policiesOf σ, PolicyStableUnderRenaming p (swapIDs a.id a'.id)) ∧
27   (∀ b ∈ rolesOf σ, ExpressionStableUnderRenaming b.trust (swapIDs a.id a'.id)) ∧
28   a.trust = a'.trust ∧
29   policiesDirectlyAttachedToRole a σ.config = policiesDirectlyAttachedToRole a' σ.config
30
31 ∨ def PolicyEquiv (σ : State) (p q : Policy) : Prop :=
32   p ∈ policiesOf σ ∧ q ∈ policiesOf σ ∧
33   (∀ r : Request, r ∈ ((eval p.allow : SafeSet) : Set Request) ↔
34    r ∈ ((eval q.allow : SafeSet) : Set Request)) ∧
35   (∀ r : Request, r ∈ ((eval p.deny : SafeSet) : Set Request) ↔
36    r ∈ ((eval q.deny : SafeSet) : Set Request)) ∧
37   (∀ o ∈ policiesOf σ, PolicyStableUnderRenaming o (swapIDs p.id q.id))
38
39 /- Reduce (pseudo-algorithm):
40   partition each kind's entities into equivalence classes, then
41   users, groups, roles → keep enough representatives, drop the rest (SELECT)
42   policies → merge each class into one (MERGE) -/
43

```

The core: Simulation

- Links states of the original system to states of the reduced one.
- Helped us prove:
 - ⇒ Soundness / Preservation — bound preserved ($n \rightarrow n$)
 - ⇐ Completeness / Reflection — bound stretches ($n \rightarrow c \cdot n$)
- Textbook definition, used unmodified.
The path induction is proved once, then reused by every reduction.

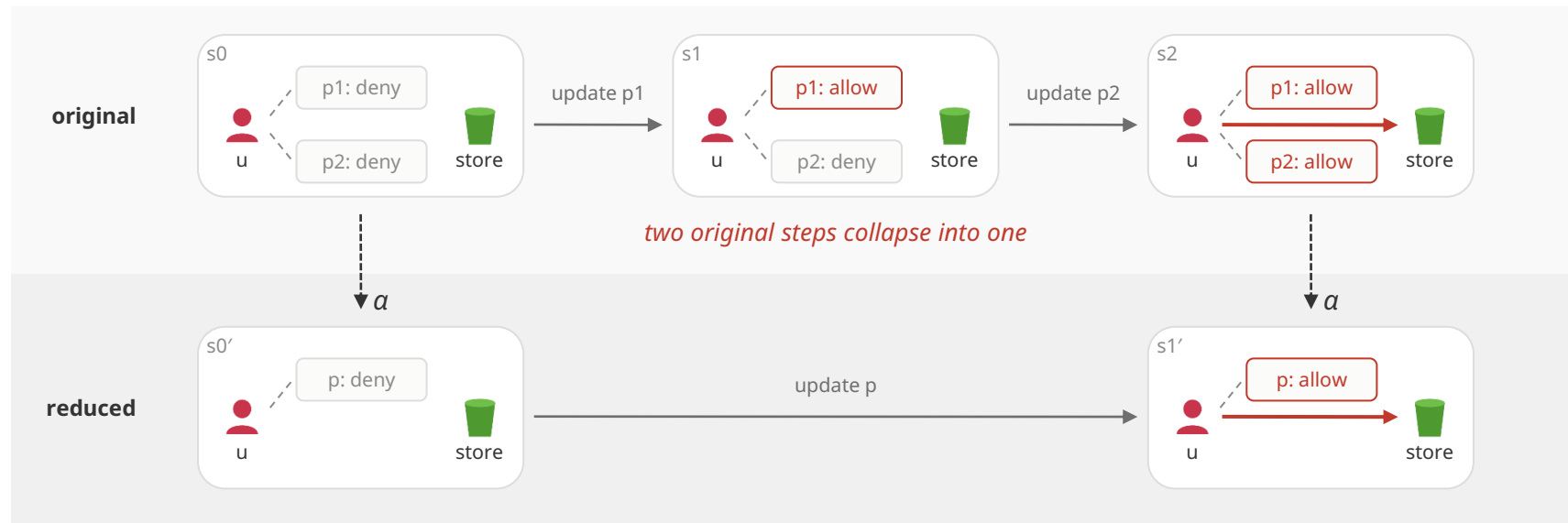
```

4
5  ∨ def IsWitnessing (R : Set Request) (Rel : State → State → Prop)
6      : Prop :=
7      | ∀ s t, Rel s t →
8      |   ∀ r, r ∈ R → r ∈ AuthZ t → ∃ r', r' ∈ AuthZ s ∧ R
9
10 ∨ def IsExtendible (TS : IAMTransitionSystem) (S1 S2 : Set Request)
11     (Rel : State → State → Prop) : Prop :=
12     | ∀ s t, Rel s t →
13     |   ∀ t' r, TS.step t r t' → r ∈ S1 →
14     |   | ∃ s' n, reachable_n_in TS S2 n s s' ∧ Rel s' t'
15
16 ∨ def IsSimulation (TS : IAMTransitionSystem) (R S1 S2 : Set Request)
17     (Rel : State → State → Prop) : Prop :=
18     | IsWitnessing R Rel ∧
19     | IsExtendible TS S1 S2 Rel
20
21 ✓ ∨ theorem PE_of_simulation
22     {TS : IAMTransitionSystem} {R S1 S2 : Set Request}
23     {Rel : State → State → Prop} {s t : State}
24     (hPE : (t, R) ∈ PE TS S1)
25     (hsim : IsSimulation TS R S1 S2 Rel)
26     (hRel : Rel s t)
27     (hcenter : centeredOn R (allIDs s.config)) :
28     (s, R) ∈ PE TS S2 :=
29     _root_.PE_of_simulation hPE hsim hRel hcenter
30

```

Careful! Mutating actions break equivalence

- Entities equivalent at state 0 need not stay equivalent along the path.
 - Example:* Users u_1 and u_2 are equivalent at $s_0 \rightarrow$ Policy p is attached to $u_1 \rightarrow$ Users u_1 and u_2 become inequivalent at s_1 .
- A BPE path could leverage this. One representative per class is almost never enough.



How many representatives are enough?

- Cannot keep only 1 representative per class.
- *Mutating actions break equivalence along the path. **We need to leave room for this to happen.***
- Proven sufficient number of representatives per entity type:

Reduction	# Representatives	at n = 5
Policies	<i>merge to</i> 1	1
Users	<i>select</i> $\min(n + 1, \text{class})$	6
Groups	<i>select</i> $\min(n + 1, \text{class})$	6
Roles	<i>select</i> $\min(n + 1, \text{class})$	6

$|\text{class}|$ = size of the equivalence class. A class smaller than the bound is kept whole.

*Upper bounds: proven sufficient, not shown necessary; tightness is still open.
Per equivalence class, not in total: 400 inequivalent users still reduce to 400.*

LESSON 3 OF 3

Proofs for insight, beyond assurance

Use proofs as a way to gain deeper understanding of systems.



What a large mechanised proof is actually like

- 40k lines of Lean code. All machine-checked.
- BPE preservation proved for each reduction (users, groups, roles, policies) with $n + 1$ representatives.
- And for the full composed sequence: policies $\rightarrow$ groups $\rightarrow$ roles $\rightarrow$ users.

Dir.	Reduction	Original path	Reduced path	Lean theorem
$\Leftrightarrow$	Policies	unbounded	unbounded	PE_reducePolicies_iff
$\Rightarrow$	Policies	$\leq n$	$\leq n$	BPE_reducePolicies_of_BPE
$\Leftarrow$	Policies	$\leq c \cdot n$	$\leq n$	BPE_linear_of_BPE_reducePolicies
$\Leftrightarrow$	Groups	$\leq n$	$\leq n$	BPE_iff_BPE_reduceGroupState
$\Leftrightarrow$	Roles	$\leq n$	$\leq n$	BPE_iff_BPE_reduceRoleState
$\Leftrightarrow$	Users	$\leq n$	$\leq n$	BPE_iff_BPE_reduceUserState
$\Rightarrow$	Pipeline	$\leq n$	$\leq n$	BPE_fullyReduce_of_BPE
$\Leftarrow$	Pipeline	$\leq c \cdot n$	$\leq n$	BPE_linear_of_BPE_fullyReduce

Did it work? Yes, it did.

- **Unblocked launch, effectively made the feature commercially viable.**
- Deployed in production in Q2 2026. Two China regions. Not generally available yet.
- Sample optimised runtimes, with the full pipeline reduction:

Dataset	# Entities	# Policies	# Policy Attachments	# Policy Statements	Baseline (s)	Optimised (s)	Speed-up
Test Account 1	25	655	6	2978	144	0.4	360×
Test Account 2	155	174	155	311	203	2.1	97×
Test Account 3	192	42	38	216	131	2	65×
Test Account 4	210	150	247	374	332	5.8	57×
Test Account 5	746	922	2351	3425	3160	1.2	2600×
Test Account 6	1263	87	76	132	3000	2.3	1300×
Test Account 7	1612	546	3015	762	OOM	17.1	—

Takeaways

- 1 Let the problem drive the solution
- 2 Textbook theory pays off
- 3 Proofs for insight, beyond assurance

